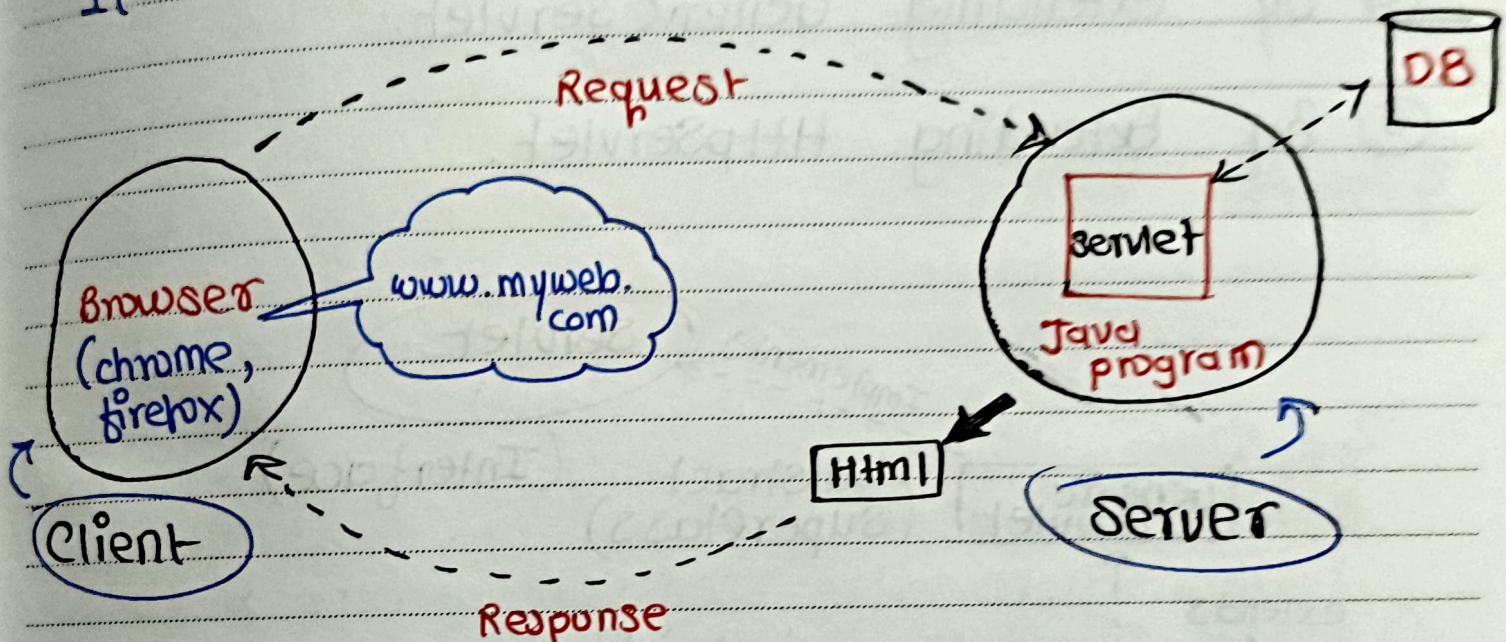
Date:

* Servlet :→

A servlet is a Java program that runs on server and handles requests from a client (like a browser).

** It is used to create dynamic web applica?



Lifecycle of a Servlet :→

Every servlet has a lifecycle, managed by the servlet container (like Apache Tomcat).

1. Loading the class.
2. Creating an object.
3. Calling init().
4. Calling service() on every request.
5. Calling destroy() when shutting down.

Date:

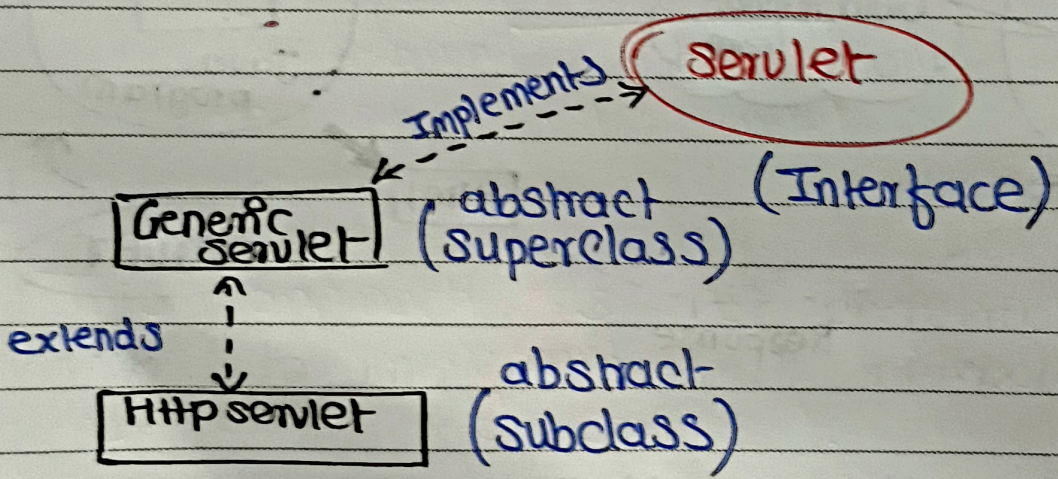
Ways to Create Servlet :→

There are 3 types/ways to create Servlet

① By Implementing Servlet Interface.

② By Extending GenericServlet.

③ By Extending HttpServlet.



Book

Date :

① By Implementing Servlet Interface :→

This is the most basic way.

Example :→

```
import javax.servlet.*;
import java.io.*;

public class MyServlet implements Servlet{

    ServletConfig config;

    public void init(ServletConfig config){
      //called once when servlet is created.
        this.config = config;
        System.out.println("init method called");
    }

    public void service(ServletRequest req,
//called for each req.  ServletResponse res)
            throws ServletException, IOException{

        res.setContentType("text/html");

        PrintWriter out = res.getWriter();

        out.print("<h1> Hello from servlet </h1>");
    }
```

Date: ..... DM DT DW DT DF DS DS

```
public void destroy() {
    // called when servlet is destroyed
    System.out.println("destroy method called");
}

public ServletConfig getServletConfig() {
    // returns config info
    return config;
}

public String getServletInfo() {
    // returns info about servlet
    return "My first servlet";
}
```

This approach is not preferred because we need to implement all methods, even if we only need service().

② By Extending GenericServlet :→

This is an abstract class that already implements the Servlet Interface, except for service().

Example →

```
import javax.servlet.*;
import java.io.*;

public class MyServlet extends GenericServlet
{
public void service(ServletRequest req,
                    ServletResponse res)
    throws ServletException, IOException {

    res.setContentType("text/html");

    PrintWriter out = res.getWriter();

    out.print("<h1> Hello from GenericServlet
                                    </h1>");
  }
}
```

Date: ☐M ☐T ☐W ☐T ☐F ☐S ☐S

{ all the methods like init(), destroy(), getServletConfig(), getServletInfo(), service() are implemented by GenericServlet from Servlet Interface }

But service() method in GenericServlet is abstract so GenericServlet class also remains abstract. (must need to override)

In this we only need to write the service() method because it is abstract must be override

Others are non abstract methods there is not compulsion to override them

GenericServlet useful for protocols other than HTTP (like FTP, etc)

Date:

③ By extending HttpServlet :→ (Most common and Recommended).

HttpServlet is a subclass of GenericServlet and designed specifically for Http protocol (used in web applications).

Example :→

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class MyServlet extends HttpServlet
{
  public void doGet(HttpServletRequest req,
                    HttpServletResponse res)
      throws ServletException, IOException
  {

    res.setContentType("text/html");

    PrintWriter out = res.getWriter();

    out.println("<h1> Hello from HttpServlet
                 </h1>");

  }
}
```

Date:

Important Methods →

① doGet()
② doPost()
③ doPut()
④ doDelete()

HttpServlet is most Referred for web application because :

i) Direct support for Http request

ii) Clear seperation for Get and Post Handling.

iii) More features for modern web applica

What is HTTP →

HTTP (HyperText Transfer Protocol) is the communication protocol used between client (browser) and server.

Each time a user interacts with a website (like clicking a button or submitting a form), the browser sends a req to the server using Http methods

Date:

Methods in HttpServlet :→

① doGet() →

- Read data (fetch resource)
- URL links, search queries, form with method = "GET"
- sends data via URL query string.

② doPost() →

- Send data (create resource)
- Submitting forms, login, registration.
- Sends data in body (more secure).

③ doPut() →

- Update existing data
- API calls to update records.
- Not used in forms, mostly APIs.
-

Date: ☐M ☐T ☐W ☐T ☐F ☐S ☐S

④ doDelete() →

- Delete existing data.
- API calls for deletion.
- Not used in forms, mostly APIs.

Deployment of Servlet :→

i) Create servlet class.

ii) src folder (MyServlet.class)

iii) Configure in web.xml (or use @WebServlet annotation).

Example :→ (web.xml) (Deployment Descriptor)

```
<web-app>
  <servlet>
    <servlet-name> myServlet </servlet-name>
    <servlet-class> MyServlet </servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name> myservlet </servlet-name>
    <url-pattern> /hello </url-pattern>
  </servlet-mapping>
</web-app>
```

Date :

Example :→ (use annotation)

```
@WebServlet("/hello")
public class MyServlet extends HttpServlet
{
....
}
```

* What is RequestDispatcher in servlet ?

RequestDispatcher is an interface in servlet API that is used to forward a request from one servlet (or JSP) to another resource (servlet, JSP, HTML) on the server side.

It allows you to share or transfer control between server components without the client knowing.

Real life Example :

Imagine you walk into a restaurant and the waiter says:
"Let me pass your order to the kitchen".

Waiter : Servlet A
Kitchen : Servlet B
Order passed = req forwarded using RequestDispatcher.

Date: ☐M ☐T ☐W ☐T ☐F ☐S ☐S

Two Main Methods in RequestDispatcher :

① forward (request, response)

Forwards the request to another resource on the server.

② include (request, response)

includes content of another resource in the response.

Syntax to use →

```
RequestDispatcher rd = request.getRequestDispatcher
                        ("secondServlet");

rd.forward (request, response);
```

OR

```
RequestDispatcher rd = request.getRequestDispatcher
                        ("header.jsp");

rd.include (request, response);
```

What is JSP (Javaserver Pages) :→

JSP stands for Javaserver Pages. It is a server-side technology used to create dynamic web pages using Java code inside HTML

Easy to write web pages with both HTML and Java.

Automatically converted into a servlet behind the scenes by the server (like Tomcat)

It is executed, and output (usually HTML) is sent to the browser.

Example →

```
<%@ page language = "java" %>
<html>
  <head><title>Welcome</title></head>
  <body>

    <h1>Hello, <% = request.getParameter
                    ("name") %></h1>

  </body>
</html>
```

Date: ☐M ☐T ☐W ☐T ☐F ☐S ☐S

Basic syntax in JSP :→

i) <%= %> → Output expression (prints value)

ii) <% %> → Scriptlet - regular Java code

iii) <%! %> → Declaration - define methods or variables.

iv) <%@ %> → Directive - set page info

v) <jsp:include> → include another JSP file.

* Layered Architecture :→

In Advance Java, applications are typically designed using a layered architecture to separate concerns and make development and maintain easier.

These layers defines how data flows and how responsibilities are devided.

(1) Presentation Layer (Client layer/ Frontend Layer)

It interacts with the user. This layer is responsible for displaying data and capturing user input.

Technologies used →

i) JSP (Java Server Pages)

ii) Servlets

iii) HTML, CSS, JavaScript

iv) JSF (Java Server Faces)

Date: ☐M ☐T ☐W ☐T ☐F ☐S ☐S

② Controller Layer (Servlet / Request Handler)

It acts as a bridge between the Presentation layer and the Business layer.

It received the request from the user interface and forwards it to the appropriate business logic.

Example →

A servlet that received from input and calls the service class method.

③ Business Layer (Service Layer)
Logic

It contains the core logic of application (i.e "what the system does").

All rules, validations, and processing happen here

Technologies used →

i) Plain old Java Object (POJOs)

ii) Spring framework (for more complex Applications)

④ Data Access Layer (DAO Layer / Persistence Layer)

It communicates with the database. This layer contains the logic to perform CRUD operation (Create, Read, Update, Delete).

Technologies Used →

i) JDBC

ii) Hibernate (ORM)

iii) JPA (Java Persistence API)

⑤ Database Layer (Backend / Storage)

Stores the data in a structured format. This is where actual data is stored and retrieved

Examples →

i) MySQL

ii) Oracle

iii) PostgreSQL.